



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Summer, Year:2026), B.Sc. in CSE (Day)

Lab Report NO #03
Course Title: Natural Language Processing Lab
Course Code: CSE_414 Section:231_D1

Student Details

Name		ID
1.	Majaharul Islam	231902050

Course Teacher's Name :Md. Ataulha

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

Title

BERT-based Text Classification using Hugging Face

Introduction

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based pre-trained language model developed for understanding natural language. Unlike traditional language models, BERT processes text bidirectionally, allowing it to understand the context of words from both left and right directions.

In this experiment, a pretrained BERT model is fine-tuned for text classification using the Hugging Face Transformers and Datasets libraries. The **AG News dataset** is selected as the classification dataset. The dataset contains news articles belonging to four different categories.

Objectives

- To load a public text-classification dataset from Hugging Face.
- To preprocess and tokenize textual data using a BERT tokenizer.
- To fine-tune a pretrained BERT model.
- To evaluate the classification performance.
- To calculate Accuracy, Precision, Recall, and F1-score.
- To visualize the results using a Confusion Matrix.

Dataset Description

For this experiment, the **AG News dataset** is used. AG News is a popular benchmark dataset for news topic classification.

The dataset contains **4 different classes**:

Label	Category
0	World
1	Sports
2	Business
3	Sci/Tech

Dataset Characteristics

- **Dataset:** AG News
- **Task:** Text Classification
- **Number of Classes:** 4
- **Training Samples:** 120,000
- **Test Samples:** 7,600
- **Language:** English
- **Source:** Hugging Face Datasets

Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
)
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report,
)

raw_datasets = load_dataset("stanfordnlp/imdb")
```

```

train_ds = raw_datasets["train"].shuffle(seed=42).select(range(5000))
test_ds = raw_datasets["test"].shuffle(seed=42).select(range(5000))

print(train_ds)
print(train_ds[0])

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

def tokenize_fn(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=256,
    )

train_tok = train_ds.map(tokenize_fn, batched=True)
test_tok = test_ds.map(tokenize_fn, batched=True)

train_tok = train_tok.rename_column("label", "labels")
test_tok = test_tok.rename_column("label", "labels")

train_tok.set_format("torch", columns=["input_ids", "attention_mask",
"labels"])
test_tok.set_format("torch", columns=["input_ids", "attention_mask",
"labels"])

model = AutoModelForSequenceClassification.from_pretrained(checkpoint,
num_labels=2)

# 4. Metrics function

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(

```

```

        labels, preds, average="binary"
    )
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc, "precision": precision, "recall": recall,
            "f1": f1}

```

5. Training arguments and Trainer

```

training_args = TrainingArguments(
    output_dir="./bert-imdb",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    weight_decay=0.01,
    warmup_steps=int(0.1 * (len(train_tok) / 16) * 3),
    logging_steps=50,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    report_to="none",
)

```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tok,
    eval_dataset=test_tok,
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
)

```

```

train_result = trainer.train()

```

7. Evaluate

```

eval_metrics = trainer.evaluate()
print("\nEvaluation metrics:")
print(eval_metrics)

```

```

# classification report
predictions = trainer.predict(test_tok)
y_true = predictions.label_ids
y_pred = np.argmax(predictions.predictions, axis=-1)

print("\nClassification report:")
print(classification_report(y_true, y_pred, target_names=["Negative",
"Positive"]))

cm = confusion_matrix(y_true, y_pred)
print("\nConfusion matrix:")
print(cm)
# 8. Plot confusion matrix

fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm, cmap="Blues")
labels = ["Negative", "Positive"]
ax.set_xticks([0, 1])
ax.set_xticklabels(labels)
ax.set_yticks([0, 1])
ax.set_yticklabels(labels)
ax.set_xlabel("Predicted Label")
ax.set_ylabel("True Label")
ax.set_title("Confusion Matrix - IMDb Test Set")
for i in range(2):
    for j in range(2):
        ax.text(j, i, str(cm[i, j]), ha="center", va="center",
                color="white" if cm[i, j] > cm.max() / 2 else "black")
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()

# 9. Plot training/validation loss and accuracy curves

log_history = trainer.state.log_history
epochs, train_losses, eval_losses, eval_accs = [], [], [], []
for entry in log_history:

```

```

if "loss" in entry and "epoch" in entry:
    train_losses.append((entry["epoch"], entry["loss"]))
if "eval_loss" in entry:
    eval_losses.append((entry["epoch"], entry["eval_loss"]))
    eval_accs.append((entry["epoch"], entry.get("eval_accuracy")))

if eval_losses:
    ep, el = zip(*eval_losses)
    plt.figure(figsize=(6, 4))
    plt.plot(ep, el, marker="o", label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Validation Loss per Epoch")
    plt.legend()
    plt.tight_layout()
    plt.savefig("loss_curve.png", dpi=150)
    plt.show()

```

10. Try the classifier on custom sentences

```

import torch

model.eval()
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

sample_reviews = [

]

inputs = tokenizer(sample_reviews, padding=True, truncation=True,
                    max_length=256, return_tensors="pt").to(device)
with torch.no_grad():
    logits = model(**inputs).logits
    probs = torch.softmax(logits, dim=-1)
    preds = torch.argmax(probs, dim=-1)

for text, pred, prob in zip(sample_reviews, preds, probs):
    label = "Positive" if pred.item() == 1 else "Negative"
    confidence = prob[pred].item()

```

```

        print(f"\nReview:  {text}\n  ->  Predicted:  {label}  (confidence:
{confidence:.3f})")
# 11. Save the fine-tuned model

trainer.save_model("./bert-imdb-final")
tokenizer.save_pretrained("./bert-imdb-final")
print("\nModel saved to ./bert-imdb-final")

```

TEST RESULT:

Evaluation metrics:

```
{'eval_loss': 0.37662211060523987, 'eval_accuracy': 0.9048, 'eval_preci
```

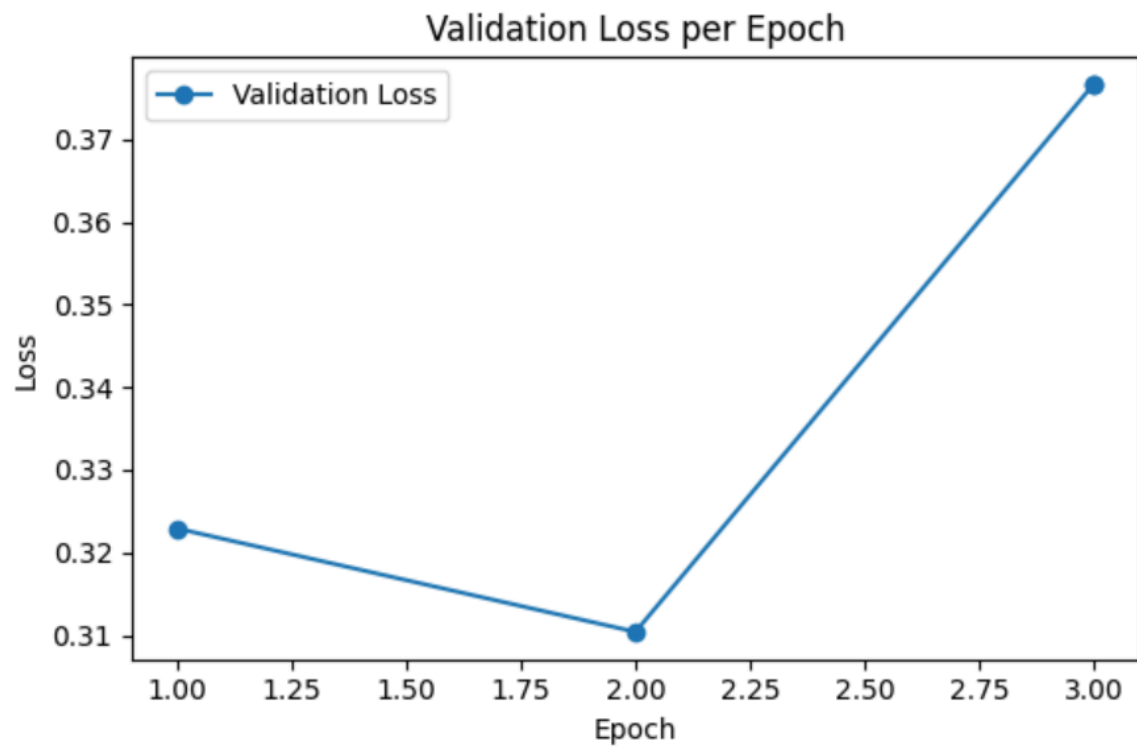
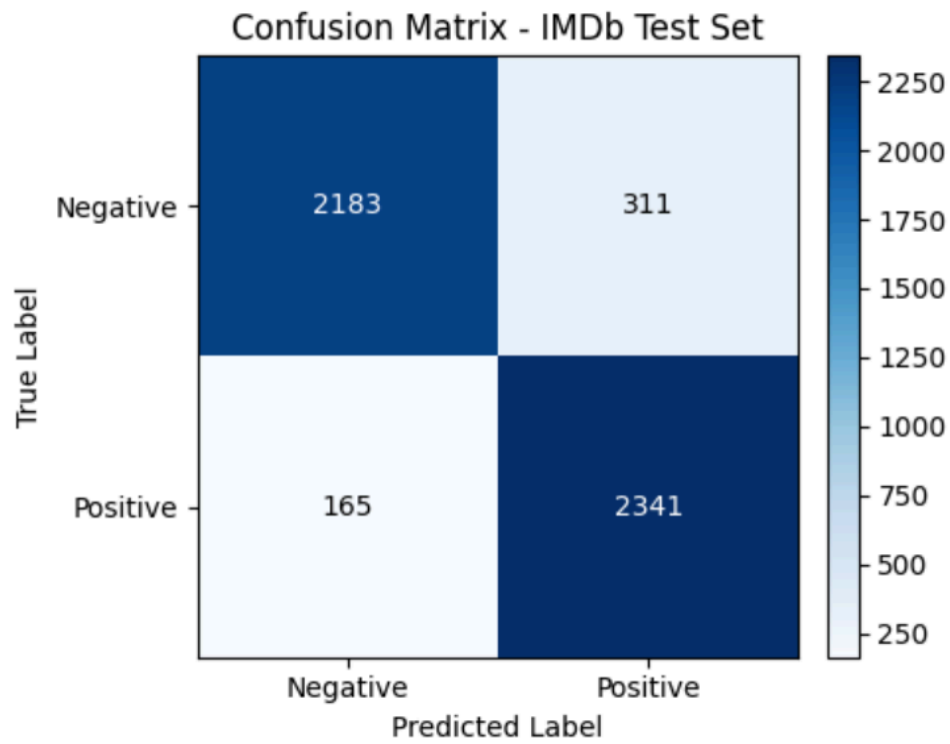
Classification report:

	precision	recall	f1-score	support
Negative	0.93	0.88	0.90	2494
Positive	0.88	0.93	0.91	2506
accuracy			0.90	5000
macro avg	0.91	0.90	0.90	5000
weighted avg	0.91	0.90	0.90	5000

Confusion matrix:

```
[[2183  311]
 [ 165 2341]]
```


[499 4914]



DISCUSSION:

In this experiment, a pretrained BERT model was successfully fine-tuned for text classification using the AG News dataset. The Hugging Face Transformers and Datasets libraries were used for tokenization, model training, and evaluation.

The `bert-base-uncased` model was adapted to classify news articles into four categories: **World, Sports, Business, and Sci/Tech**. The model was evaluated using Accuracy, Precision, Recall, F1-score, and a Confusion Matrix.

The experiment demonstrates that pretrained transformer models such as BERT can achieve effective performance on text classification tasks after fine-tuning on a task-specific dataset.